

Infosys / HackWithInfy DSA Prep — Pattern-Based Guide

Index (Learning Order)

Module 1 — Arrays & Greedy

1. Maximum Subarray Sum (Kadane's Algorithm)
2. Greedy Interval Scheduling (Maximize non-overlapping tasks)
3. Minimum Operations to Make Array Non-Decreasing

Module 2 — Prefix Sum & Pre-computation 4. Range Sum Queries (Prefix Sum) 5. Subarray Sum Equals K (Prefix Sum + HashMap)

Module 3 — Sliding Window & Hashing 6. Longest Substring Without Repeating Characters 7. Longest Subarray with Sum $\leq K$ (Sliding Window)

Module 4 — Binary Search 8. Binary Search on Answer (Minimize the Maximum / Capacity to Ship Packages)

Module 5 — Dynamic Programming 9. Longest Common Subsequence (LCS) 10. Edit Distance (Min Cost String Transform) 11. Longest Increasing Subsequence ($O(N \log N)$) 12. Count Ways to Reach Target Sum (Coin Change variant) 13. 0/1 Knapsack (Max Value Under Constraint)

Module 6 — Trees 14. Diameter of a Binary Tree 15. Lowest Common Ancestor (LCA) 16. Subtree Sum / Tree DP 17. Maximum Path Sum in a Tree

Module 7 — Graphs 18. DSU / Union-Find (Connectivity Queries) 19. Cycle Detection (Directed & Undirected) 20. Dijkstra's Shortest Path 21. Minimum Spanning Tree (Kruskal + DSU) 22. Bipartite Graph Check 23. Topological Sort

Module 8 — Advanced Data Structures 24. Segment Tree (Range Sum + Point Update) 25. Fenwick Tree / BIT (Prefix Sum + Point Update)

Module 9 — String DP 26. Longest Palindromic Subsequence

Format for every problem below: Problem Statement → Sample Test Case → Java Solution.

1. Maximum Subarray Sum (Kadane's Algorithm)

Problem Statement: Given an integer array, find the contiguous subarray (containing at least one number) which has the largest sum, and return that sum.

Sample Test Case:

```
Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]
Output: 6
Explanation: [4, -1, 2, 1] has the largest sum = 6
```

Java Solution:

```
public class MaxSubarraySum {
    public static int maxSubArray(int[] nums) {
        int maxSoFar = nums[0];
        int maxEndingHere = nums[0];
        for (int i = 1; i < nums.length; i++) {
            maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);
            maxSoFar = Math.max(maxSoFar, maxEndingHere);
        }
        return maxSoFar;
    }

    public static void main(String[] args) {
        int[] nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
        System.out.println(maxSubArray(nums)); // 6
    }
}
```

2. Greedy Interval Scheduling

Problem Statement: Given N intervals with start and end times, find the maximum number of non-overlapping intervals that can be selected.

Sample Test Case:

Input: intervals = [[1,3],[2,4],[3,5],[7,9]]
Output: 3
Explanation: Pick [1,3],[3,5],[7,9]

Java Solution:

```
import java.util.*;

public class IntervalScheduling {
    public static int maxNonOverlapping(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // sort by end time
        int count = 0;
        int lastEnd = Integer.MIN_VALUE;
        for (int[] interval : intervals) {
            if (interval[0] >= lastEnd) {
                count++;
                lastEnd = interval[1];
            }
        }
        return count;
    }

    public static void main(String[] args) {
        int[][] intervals = {{1,3},{2,4},{3,5},{7,9}};
        System.out.println(maxNonOverlapping(intervals)); // 3
    }
}
```

3. Minimum Operations to Make Array Non-Decreasing

Problem Statement: Given an array, in one operation you can increment any element by 1. Find the minimum number of operations to make the array non-decreasing.

Sample Test Case:

Input: [3, 1, 2, 5, 4]
Output: 4
Explanation: Make it [3,3,3,5,5] -> ops = 2+1+0+0+1 = 4

Java Solution:

```

public class MinOpsNonDecreasing {
    public static int minOperations(int[] arr) {
        int ops = 0;
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < arr[i - 1]) {
                ops += (arr[i - 1] - arr[i]);
                arr[i] = arr[i - 1];
            }
        }
        return ops;
    }

    public static void main(String[] args) {
        int[] arr = {3, 1, 2, 5, 4};
        System.out.println(minOperations(arr)); // 4
    }
}

```

4. Range Sum Queries (Prefix Sum)

Problem Statement: Given an array and multiple queries of type (l, r), return the sum of elements from index l to r (inclusive) efficiently.

Sample Test Case:

Input: arr = [2, 4, 5, 1, 3], queries = [[0,2],[1,3],[0,4]]
Output: [11, 10, 15]

Java Solution:

```

public class RangeSumQuery {
    public static int[] rangeSum(int[] arr, int[][] queries) {
        int n = arr.length;
        int[] prefix = new int[n + 1];
        for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + arr[i];

        int[] result = new int[queries.length];
        for (int i = 0; i < queries.length; i++) {
            int l = queries[i][0], r = queries[i][1];
            result[i] = prefix[r + 1] - prefix[l];
        }
        return result;
    }

    public static void main(String[] args) {
        int[] arr = {2, 4, 5, 1, 3};
        int[][] queries = {{0,2},{1,3},{0,4}};
        int[] res = rangeSum(arr, queries);
        for (int r : res) System.out.print(r + " "); // 11 10 15
    }
}

```

5. Subarray Sum Equals K (Prefix Sum + HashMap)

Problem Statement: Given an array of integers and a target sum K, find the total number of contiguous subarrays whose sum equals K.

Sample Test Case:

Input: nums = [1, 1, 1], k = 2
Output: 2

Java Solution:

```
import java.util.*;

public class SubarraySumEqualsK {
    public static int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> prefixCount = new HashMap<>();
        prefixCount.put(0, 1);
        int sum = 0, count = 0;
        for (int num : nums) {
            sum += num;
            count += prefixCount.getOrDefault(sum - k, 0);
            prefixCount.put(sum, prefixCount.getOrDefault(sum, 0) + 1);
        }
        return count;
    }

    public static void main(String[] args) {
        int[] nums = {1, 1, 1};
        System.out.println(subarraySum(nums, 2)); // 2
    }
}
```

6. Longest Substring Without Repeating Characters

Problem Statement: Given a string, find the length of the longest substring without repeating characters.

Sample Test Case:

Input: "abcabcbb"
 Output: 3
 Explanation: "abc" has length 3

Java Solution:

```
import java.util.*;

public class LongestUniqueSubstring {
    public static int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> lastSeen = new HashMap<>();
        int maxLen = 0, start = 0;
        for (int end = 0; end < s.length(); end++) {
            char c = s.charAt(end);
            if (lastSeen.containsKey(c) && lastSeen.get(c) >= start) {
                start = lastSeen.get(c) + 1;
            }
            lastSeen.put(c, end);
            maxLen = Math.max(maxLen, end - start + 1);
        }
        return maxLen;
    }

    public static void main(String[] args) {
        System.out.println(lengthOfLongestSubstring("abcabcbb")); // 3
    }
}
```

7. Longest Subarray with Sum $\leq K$ (Sliding Window)

Problem Statement: Given an array of positive integers and a value K, find the length of the longest subarray whose sum is less than or equal to K.

Sample Test Case:

Input: arr = [2, 1, 3, 4, 1, 1], k = 7

Output: 4

Explanation: [3,4,1,1] wait check -> longest is [1,3,4]... actual longest valid = [4,1,1,?]

Correct: subarray [1,1] extended -> longest window with sum<=7 is [3,4] len2 or [4,1,1] sum6 len3 or [1,3,4] wait order matters, using

Actual answer for this array/order: 3

Java Solution:

```
public class LongestSubarraySumLEK {  
    public static int longestSubarray(int[] arr, int k) {  
        int left = 0, sum = 0, maxLen = 0;  
        for (int right = 0; right < arr.length; right++) {  
            sum += arr[right];  
            while (sum > k) {  
                sum -= arr[left];  
                left++;  
            }  
            maxLen = Math.max(maxLen, right - left + 1);  
        }  
        return maxLen;  
    }  
  
    public static void main(String[] args) {  
        int[] arr = {2, 1, 3, 4, 1, 1};  
        System.out.println(longestSubarray(arr, 7)); // 3  
    }  
}
```

8. Binary Search on Answer — Capacity to Ship Packages

Problem Statement: Given weights of packages and a number of days D, find the minimum ship capacity such that all packages can be shipped within D days (packages must be shipped in order, each day's total weight $\leq$ capacity).

Sample Test Case:

Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5

Output: 15

Java Solution:

```

public class ShipWithinDays {
    public static int shipWithinDays(int[] weights, int days) {
        int lo = 0, hi = 0;
        for (int w : weights) { lo = Math.max(lo, w); hi += w; }

        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (canShip(weights, days, mid)) hi = mid;
            else lo = mid + 1;
        }
        return lo;
    }

    private static boolean canShip(int[] weights, int days, int capacity) {
        int daysNeeded = 1, currentLoad = 0;
        for (int w : weights) {
            if (currentLoad + w > capacity) {
                daysNeeded++;
                currentLoad = 0;
            }
            currentLoad += w;
        }
        return daysNeeded <= days;
    }

    public static void main(String[] args) {
        int[] weights = {1,2,3,4,5,6,7,8,9,10};
        System.out.println(shipWithinDays(weights, 5)); // 15
    }
}

```

9. Longest Common Subsequence (LCS)

Problem Statement: Given two strings, find the length of their longest common subsequence.

Sample Test Case:

Input: text1 = "abcde", text2 = "ace"
 Output: 3
 Explanation: LCS is "ace"

Java Solution:

```

public class LongestCommonSubsequence {
    public static int lcs(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i - 1) == text2.charAt(j - 1))
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                else
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }

    public static void main(String[] args) {
        System.out.println(lcs("abcde", "ace")); // 3
    }
}

```

10. Edit Distance (Min Cost String Transform)

Problem Statement: Given two strings, find the minimum number of operations (insert, delete, replace) required to convert one string into the other.

Sample Test Case:

Input: word1 = "horse", word2 = "ros"
Output: 3

Java Solution:

```
public class EditDistance {  
    public static int minDistance(String word1, String word2) {  
        int m = word1.length(), n = word2.length();  
        int[][] dp = new int[m + 1][n + 1];  
        for (int i = 0; i <= m; i++) dp[i][0] = i;  
        for (int j = 0; j <= n; j++) dp[0][j] = j;  
  
        for (int i = 1; i <= m; i++) {  
            for (int j = 1; j <= n; j++) {  
                if (word1.charAt(i - 1) == word2.charAt(j - 1))  
                    dp[i][j] = dp[i - 1][j - 1];  
                else  
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],  
                                           Math.min(dp[i - 1][j], dp[i][j - 1]));  
            }  
        }  
        return dp[m][n];  
    }  
  
    public static void main(String[] args) {  
        System.out.println(minDistance("horse", "ros")); // 3  
    }  
}
```

11. Longest Increasing Subsequence — $O(N \log N)$

Problem Statement: Given an array of integers, find the length of the longest strictly increasing subsequence.

Sample Test Case:

Input: [10, 9, 2, 5, 3, 7, 101, 18]
Output: 4
Explanation: [2, 3, 7, 101]

Java Solution:

```
import java.util.*;

public class LongestIncreasingSubsequence {
    public static int lengthOfLIS(int[] nums) {
        List<Integer> tails = new ArrayList<>();
        for (int num : nums) {
            int pos = Collections.binarySearch(tails, num);
            if (pos < 0) pos = -(pos + 1);
            if (pos == tails.size()) tails.add(num);
            else tails.set(pos, num);
        }
        return tails.size();
    }

    public static void main(String[] args) {
        int[] nums = {10, 9, 2, 5, 3, 7, 101, 18};
        System.out.println(lengthOfLIS(nums)); // 4
    }
}
```

12. Count Ways to Reach Target Sum (Coin Change — Combinations)

Problem Statement: Given an array of coin denominations and a target amount, count the number of ways to make up that amount using the coins (unlimited supply, order doesn't matter).

Sample Test Case:

```
Input: coins = [1, 2, 3], amount = 4
Output: 4
Explanation: {1,1,1,1},{1,1,2},{2,2},{1,3}
```

Java Solution:

```
public class CoinChangeWays {
    public static int change(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        dp[0] = 1;
        for (int coin : coins) {
            for (int i = coin; i <= amount; i++) {
                dp[i] += dp[i - coin];
            }
        }
        return dp[amount];
    }

    public static void main(String[] args) {
        int[] coins = {1, 2, 3};
        System.out.println(change(coins, 4)); // 4
    }
}
```

13. 0/1 Knapsack (Max Value Under Constraint)

Problem Statement: Given weights and values of N items and a knapsack capacity W, find the maximum value that can be obtained by selecting items such that total weight $\leq W$ (each item can be used at most once).

Sample Test Case:

Input: weights=[1,3,4,5], values=[1,4,5,7], W=7
Output: 9
Explanation: Pick items with weight 3 and 4 -> value 4+5=9

Java Solution:

```
public class Knapsack01 {  
    public static int knapsack(int[] weights, int[] values, int W) {  
        int n = weights.length;  
        int[][] dp = new int[n + 1][W + 1];  
        for (int i = 1; i <= n; i++) {  
            for (int w = 0; w <= W; w++) {  
                dp[i][w] = dp[i - 1][w];  
                if (weights[i - 1] <= w) {  
                    dp[i][w] = Math.max(dp[i][w],  
                                         dp[i - 1][w - weights[i - 1]] + values[i - 1]);  
                }  
            }  
        }  
        return dp[n][W];  
    }  
  
    public static void main(String[] args) {  
        int[] weights = {1,3,4,5};  
        int[] values = {1,4,5,7};  
        System.out.println(knapsack(weights, values, 7)); // 9  
    }  
}
```

14. Diameter of a Binary Tree

Problem Statement: Given a binary tree, find the length of the diameter — the number of edges on the longest path between any two nodes.

Sample Test Case:

Input tree:

```
      1  
     / \  
    2   3  
   / \  
  4   5
```

Output: 3
Explanation: path 4-2-5 or 4-2-1-3 -> longest = 3 edges (4-2-1-3)

Java Solution:

```

public class TreeDiameter {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    static int diameter = 0;

    public static int diameterOfBinaryTree(TreeNode root) {
        diameter = 0;
        height(root);
        return diameter;
    }

    private static int height(TreeNode node) {
        if (node == null) return 0;
        int leftHeight = height(node.left);
        int rightHeight = height(node.right);
        diameter = Math.max(diameter, leftHeight + rightHeight);
        return 1 + Math.max(leftHeight, rightHeight);
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(1);
        root.left = new TreeNode(2);
        root.right = new TreeNode(3);
        root.left.left = new TreeNode(4);
        root.left.right = new TreeNode(5);
        System.out.println(diameterOfBinaryTree(root)); // 3
    }
}

```

15. Lowest Common Ancestor (LCA)

Problem Statement: Given a binary tree and two node values, find their lowest common ancestor.

Sample Test Case:

```

Input tree:
      3
     / \
    5   1
   / \ / \
  6  2 0  8
   / \
  7   4
p = 5, q = 1
Output: 3

```

Java Solution:

```

public class LowestCommonAncestor {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    public static TreeNode lca(TreeNode root, int p, int q) {
        if (root == null || root.val == p || root.val == q) return root;
        TreeNode left = lca(root.left, p, q);
        TreeNode right = lca(root.right, p, q);
        if (left != null && right != null) return root;
        return left != null ? left : right;
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(3);
        root.left = new TreeNode(5);
        root.right = new TreeNode(1);
        root.left.left = new TreeNode(6);
        root.left.right = new TreeNode(2);
        root.right.left = new TreeNode(0);
        root.right.right = new TreeNode(8);
        root.left.right.left = new TreeNode(7);
        root.left.right.right = new TreeNode(4);

        TreeNode result = lca(root, 5, 1);
        System.out.println(result.val); // 3
    }
}

```

16. Subtree Sum / Tree DP

Problem Statement: Given a tree with integer node values, compute the sum of all nodes in the subtree rooted at each node.

Sample Test Case:

```

Input tree:
      1
     / \
    2   3
   / \
  4   5

Output subtree sums: node1=15, node2=11, node3=3, node4=4, node5=5

```

Java Solution:

```

import java.util.*;

public class SubtreeSum {
    static Map<Integer, List<Integer>> tree = new HashMap<>();
    static int[] values;
    static int[] subtreeSum;

    public static void dfs(int node, int parent) {
        subtreeSum[node] = values[node];
        for (int child : tree.getOrDefault(node, new ArrayList<>())) {
            if (child != parent) {
                dfs(child, node);
                subtreeSum[node] += subtreeSum[child];
            }
        }
    }

    public static void main(String[] args) {
        values = new int[]{0, 1, 2, 3, 4, 5}; // 1-indexed, index0 unused
        tree.put(1, Arrays.asList(2, 3));
        tree.put(2, Arrays.asList(1, 4, 5));
        tree.put(3, Arrays.asList(1));
        tree.put(4, Arrays.asList(2));
        tree.put(5, Arrays.asList(2));

        subtreeSum = new int[6];
        dfs(1, -1);

        for (int i = 1; i <= 5; i++) {
            System.out.println("Node " + i + " subtree sum = " + subtreeSum[i]);
        }
    }
}

```

17. Maximum Path Sum in a Tree

Problem Statement: Given a binary tree with possibly negative values, find the maximum path sum. A path can start and end at any node (does not need to pass through the root).

Sample Test Case:

```

Input tree:
  -10
 /  \
9    20
 /  \
15   7
Output: 42
Explanation: path 15 -> 20 -> 7

```

Java Solution:

```

public class MaxPathSumTree {
    static class TreeNode {
        int val;
        TreeNode left, right;
        TreeNode(int val) { this.val = val; }
    }

    static int maxSum = Integer.MIN_VALUE;

    public static int maxPathSum(TreeNode root) {
        maxSum = Integer.MIN_VALUE;
        maxGain(root);
        return maxSum;
    }

    private static int maxGain(TreeNode node) {
        if (node == null) return 0;
        int leftGain = Math.max(maxGain(node.left), 0);
        int rightGain = Math.max(maxGain(node.right), 0);
        maxSum = Math.max(maxSum, node.val + leftGain + rightGain);
        return node.val + Math.max(leftGain, rightGain);
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(-10);
        root.left = new TreeNode(9);
        root.right = new TreeNode(20);
        root.right.left = new TreeNode(15);
        root.right.right = new TreeNode(7);
        System.out.println(maxPathSum(root)); // 42
    }
}

```

18. DSU / Union-Find (Connectivity Queries)

Problem Statement: Given N nodes and a list of union operations, answer queries asking whether two nodes are connected.

Sample Test Case:

```

Input: n=5, unions=[[0,1],[1,2],[3,4]]
Query: connected(0,2) -> true
Query: connected(0,3) -> false

```

Java Solution:

```
public class DisjointSetUnion {
    int[] parent, rank_;

    public DisjointSetUnion(int n) {
        parent = new int[n];
        rank_ = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path compression
        return parent[x];
    }

    public void union(int x, int y) {
        int rootX = find(x), rootY = find(y);
        if (rootX == rootY) return;
        if (rank_[rootX] < rank_[rootY]) { int t = rootX; rootX = rootY; rootY = t; }
        parent[rootY] = rootX;
        if (rank_[rootX] == rank_[rootY]) rank_[rootX]++;
    }

    public boolean connected(int x, int y) {
        return find(x) == find(y);
    }

    public static void main(String[] args) {
        DisjointSetUnion dsu = new DisjointSetUnion(5);
        dsu.union(0, 1);
        dsu.union(1, 2);
        dsu.union(3, 4);
        System.out.println(dsu.connected(0, 2)); // true
        System.out.println(dsu.connected(0, 3)); // false
    }
}
```

19. Cycle Detection (Directed & Undirected Graph)

Problem Statement: Given a directed graph, determine whether it contains a cycle.

Sample Test Case:

Input: n=4, edges=[[0,1],[1,2],[2,3],[3,1]]
Output: true (cycle: 1->2->3->1)

Java Solution:

```

import java.util.*;

public class CycleDetectionDirected {
    public static boolean hasCycle(int n, List<List<Integer>> adj) {
        int[] visited = new int[n]; // 0=unvisited, 1=visiting, 2=done
        for (int i = 0; i < n; i++) {
            if (visited[i] == 0 && dfs(i, adj, visited)) return true;
        }
        return false;
    }

    private static boolean dfs(int node, List<List<Integer>> adj, int[] visited) {
        visited[node] = 1;
        for (int neighbor : adj.get(node)) {
            if (visited[neighbor] == 1) return true;
            if (visited[neighbor] == 0 && dfs(neighbor, adj, visited)) return true;
        }
        visited[node] = 2;
        return false;
    }

    public static void main(String[] args) {
        int n = 4;
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        int[][] edges = {{0,1},{1,2},{2,3},{3,1}};
        for (int[] e : edges) adj.get(e[0]).add(e[1]);

        System.out.println(hasCycle(n, adj)); // true
    }
}

```

20. Dijkstra's Shortest Path

Problem Statement: Given a weighted graph with non-negative edge weights and a source node, find the shortest distance from the source to all other nodes.

Sample Test Case:

Input: n=5, edges=[[0,1,2],[0,2,4],[1,2,1],[1,3,7],[2,4,3],[3,4,1]], source=0
 Output distances: [0, 2, 3, 9, 6]

Java Solution:

```

import java.util.*;

public class DijkstraShortestPath {
    public static int[] dijkstra(int n, int[][] edges, int source) {
        List<List<int[]>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        for (int[] e : edges) {
            adj.get(e[0]).add(new int[]{e[1], e[2]});
            adj.get(e[1]).add(new int[]{e[0], e[2]}); // undirected
        }

        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[source] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.add(new int[]{source, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int node = curr[0], d = curr[1];
            if (d > dist[node]) continue;
            for (int[] next : adj.get(node)) {
                int neighbor = next[0], weight = next[1];
                if (dist[node] + weight < dist[neighbor]) {
                    dist[neighbor] = dist[node] + weight;
                    pq.add(new int[]{neighbor, dist[neighbor]});
                }
            }
        }
        return dist;
    }

    public static void main(String[] args) {
        int[][] edges = {{0,1,2},{0,2,4},{1,2,1},{1,3,7},{2,4,3},{3,4,1}};
        int[] dist = dijkstra(5, edges, 0);
        System.out.println(Arrays.toString(dist)); // [0, 2, 3, 9, 6]
    }
}

```

21. Minimum Spanning Tree (Kruskal + DSU)

Problem Statement: Given a weighted undirected graph, find the minimum cost to connect all nodes (Minimum Spanning Tree weight).

Sample Test Case:

Input: n=4, edges=[[0,1,10],[0,2,6],[0,3,5],[1,3,15],[2,3,4]]
Output: 19

Java Solution:


```

import java.util.*;

public class MinimumSpanningTreeKruskal {
    static int[] parent;

    static int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    static boolean union(int x, int y) {
        int rx = find(x), ry = find(y);
        if (rx == ry) return false;
        parent[rx] = ry;
        return true;
    }

    public static int minCostConnect(int n, int[][] edges) {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;

        Arrays.sort(edges, (a, b) -> a[2] - b[2]);
        int totalCost = 0, edgesUsed = 0;

        for (int[] edge : edges) {
            if (union(edge[0], edge[1])) {
                totalCost += edge[2];
                edgesUsed++;
                if (edgesUsed == n - 1) break;
            }
        }
        return totalCost;
    }

    public static void main(String[] args) {
        int[][] edges = {{0,1,10},{0,2,6},{0,3,5},{1,3,15},{2,3,4}};
        System.out.println(minCostConnect(4, edges)); // 19
    }
}

```

22. Bipartite Graph Check

Problem Statement: Given an undirected graph, determine whether it can be colored using two colors such that no two adjacent nodes share the same color (i.e., check if it is bipartite).

Sample Test Case:

Input: n=4, edges=[[0,1],[1,2],[2,3],[3,0]]
Output: true

Java Solution:

```

import java.util.*;

public class BipartiteCheck {
    public static boolean isBipartite(int n, List<List<Integer>> adj) {
        int[] color = new int[n];
        Arrays.fill(color, -1);

        for (int i = 0; i < n; i++) {
            if (color[i] == -1) {
                Queue<Integer> queue = new LinkedList<>();
                queue.add(i);
                color[i] = 0;
                while (!queue.isEmpty()) {
                    int node = queue.poll();
                    for (int neighbor : adj.get(node)) {
                        if (color[neighbor] == -1) {
                            color[neighbor] = 1 - color[node];
                            queue.add(neighbor);
                        } else if (color[neighbor] == color[node]) {
                            return false;
                        }
                    }
                }
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int n = 4;
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        int[][] edges = {{0,1},{1,2},{2,3},{3,0}};
        for (int[] e : edges) {
            adj.get(e[0]).add(e[1]);
            adj.get(e[1]).add(e[0]);
        }
        System.out.println(isBipartite(n, adj)); // true
    }
}

```

23. Topological Sort

Problem Statement: Given a directed acyclic graph (DAG), return a valid topological ordering of its nodes.

Sample Test Case:

Input: n=6, edges=[[5,2],[5,0],[4,0],[4,1],[2,3],[3,1]]
 Output (one valid order): [4, 5, 2, 3, 1, 0]

Java Solution:

```

import java.util.*;

public class TopologicalSort {
    public static List<Integer> topoSort(int n, List<List<Integer>> adj) {
        int[] inDegree = new int[n];
        for (List<Integer> neighbors : adj) {
            for (int v : neighbors) inDegree[v]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < n; i++) if (inDegree[i] == 0) queue.add(i);

        List<Integer> order = new ArrayList<>();
        while (!queue.isEmpty()) {
            int node = queue.poll();
            order.add(node);
            for (int neighbor : adj.get(node)) {
                inDegree[neighbor]--;
                if (inDegree[neighbor] == 0) queue.add(neighbor);
            }
        }
        return order;
    }

    public static void main(String[] args) {
        int n = 6;
        List<List<Integer>> adj = new ArrayList<>();
        for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
        int[][] edges = {{5,2},{5,0},{4,0},{4,1},{2,3},{3,1}};
        for (int[] e : edges) adj.get(e[0]).add(e[1]);

        System.out.println(topoSort(n, adj)); // e.g. [4, 5, 2, 0, 3, 1]
    }
}

```

24. Segment Tree (Range Sum + Point Update)

Problem Statement: Given an array, support two operations efficiently: (1) update the value at an index, (2) query the sum of a range [l, r].

Sample Test Case:

```

Input: arr = [1, 3, 5, 7, 9, 11]
Query sum(1, 3) -> 15
Update index 1 to 10
Query sum(1, 3) -> 22

```

Java Solution:

```

public class SegmentTree {
    int[] tree;
    int n;

    public SegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n];
        build(arr, 0, 0, n - 1);
    }

    private void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
            return;
        }
        int mid = (start + end) / 2;
        build(arr, 2 * node + 1, start, mid);
        build(arr, 2 * node + 2, mid + 1, end);
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }

    public void update(int index, int value) {
        update(0, 0, n - 1, index, value);
    }

    private void update(int node, int start, int end, int index, int value) {
        if (start == end) {
            tree[node] = value;
            return;
        }
        int mid = (start + end) / 2;
        if (index <= mid) update(2 * node + 1, start, mid, index, value);
        else update(2 * node + 2, mid + 1, end, index, value);
        tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
    }

    public int query(int l, int r) {
        return query(0, 0, n - 1, l, r);
    }

    private int query(int node, int start, int end, int l, int r) {
        if (r < start || end < l) return 0;
        if (l <= start && end <= r) return tree[node];
        int mid = (start + end) / 2;
        return query(2 * node + 1, start, mid, l, r) +
            query(2 * node + 2, mid + 1, end, l, r);
    }

    public static void main(String[] args) {
        int[] arr = {1, 3, 5, 7, 9, 11};
        SegmentTree st = new SegmentTree(arr);
        System.out.println(st.query(1, 3)); // 15
        st.update(1, 10);
        System.out.println(st.query(1, 3)); // 22
    }
}

```

25. Fenwick Tree / BIT (Prefix Sum + Point Update)

Problem Statement: Given an array, support point updates and prefix sum queries efficiently using a Binary Indexed Tree.

Sample Test Case:

Input: arr = [1, 3, 5, 7, 9, 11]
Query prefixSum(4) -> 25 (1+3+5+7+9)
Update index 2 to add +6 (arr[2] becomes 11)
Query prefixSum(4) -> 31

Java Solution:

```
public class FenwickTree {
    int[] bit;
    int n;

    public FenwickTree(int[] arr) {
        n = arr.length;
        bit = new int[n + 1];
        for (int i = 0; i < n; i++) update(i, arr[i]);
    }

    public void update(int index, int delta) {
        for (int i = index + 1; i <= n; i += i & (-i)) {
            bit[i] += delta;
        }
    }

    public int prefixSum(int index) {
        int sum = 0;
        for (int i = index + 1; i > 0; i -= i & (-i)) {
            sum += bit[i];
        }
        return sum;
    }

    public static void main(String[] args) {
        int[] arr = {1, 3, 5, 7, 9, 11};
        FenwickTree ft = new FenwickTree(arr);
        System.out.println(ft.prefixSum(4)); // 25 (sum of indices 0..4)
        ft.update(2, 6); // add 6 to index 2
        System.out.println(ft.prefixSum(4)); // 31
    }
}
```

26. Longest Palindromic Subsequence (String DP)

Problem Statement: Given a string, find the length of the longest palindromic subsequence.

Sample Test Case:

Input: "bbbab"
Output: 4
Explanation: "bbbb" is the longest palindromic subsequence

Java Solution:

```
public class LongestPalindromicSubsequence {
    public static int longestPalinSubseq(String s) {
        int n = s.length();
        int[][] dp = new int[n][n];
        for (int i = n - 1; i >= 0; i--) {
            dp[i][i] = 1;
            for (int j = i + 1; j < n; j++) {
                if (s.charAt(i) == s.charAt(j)) {
                    dp[i][j] = dp[i + 1][j - 1] + 2;
                } else {
                    dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
                }
            }
        }
        return dp[0][n - 1];
    }

    public static void main(String[] args) {
        System.out.println(longestPalinSubseq("bbbab")); // 4
    }
}
```

Study Tips

- Do the modules **in order** — each builds intuition needed for the next (prefix sum → sliding window → DP; DSU → Kruskal; DFS basics → tree/graph problems).
- For every problem: solve brute force first, identify the pattern, then optimize.
- Re-implement each solution from scratch without looking, 24 hours later — that's what makes patterns stick.
- HackWithInfy rounds tend to mix Easy (arrays/prefix), Medium (DP/graphs), and Hard (advanced DS/trees) — so don't skip Module 8.